

Prompt-State Machines: Japanese Companion Translation

Takuya Sogawa

プロンプトステートマシン

ILA 抽象化規律を用いた LLM 推論の構造化

Version: v1.0.0

Type: Technical Note / ILA Supplement 2

Author: Takuya Sogawa

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: 10.5281/zenodo.20323512

License: CC BY 4.0

Abstract

本稿は、Interface-Led Architecture (ILA) および Provider-Observer-Operator 抽象化規律を LLM プロンプト設計へ応用するための Prompt-State Machine モデルを提案する。

LLM は本質的に確率的生成器であり、その内部推論過程をプロンプトだけで完全に決定論化することはできない。しかし、プロンプトを構造化することで、LLM の出力過程を明示的な役割、状態、遷移、失敗経路として誘導・制約することはできる。この意味で、プロンプトは軽量の疑似ステートマシンとして機能し得る。ただし、それはカーネルレベルで強制される決定論的オートマトンではなく、プロンプトレベルのガバナンスプロトコルである。

本稿では、Provider を前提、知識、制約、入力材料の供給状態として定義する。Observer は曖昧性、不足、矛盾、リスク、危険な仮定を観測する状態である。Operator は、Provider と Observer の結果に基づいて、変換、推論、合成、または回答生成を行う状態である。

さらに本稿は、Markdown、特に ROM 的な Markdown が、LLM にとって実用的な状態境界表現であることを論じる。見出し、YAML front matter、箇条書き、コードブロックを用いることで、プロンプトは人間にも LLM にも読みやすい構造化された境界になる。Clarify と FailClosed を明示的な遷移として組み込むことで、不明確または安全でない要求が、黙って最終回答へ昇格されることを防ぐ。また、Provider に利用可能な推論能力を宣言する Skill-Set Provider パターンを実用例として提示する。

本稿の主な貢献は、ILA の役割ベース抽象化規律をプロンプト空間へ適用し、LLM 推論を状態境界、役割責務、失敗時遷移を持つ Prompt-State Machine として構造化するための軽量のガバナンスモデルを提示する点にある。

Keywords

Interface-Led Architecture; ILA; Prompt-State Machine; Provider; Observer; Operator; LLM Reasoning; Prompt Engineering; Markdown; ROM; Fail-Closed; Clarify; Governed Reasoning; AIKernel

1. Introduction

プロンプトエンジニアリングは、しばしば「よりよい指示文を書く技術」として説明される。この説明は有用であるが、十分ではない。LLM を自律エージェント、ツール利用型アシスタント、または AIOS の一部として扱う場合、プロンプトは単なる命令文ではなく、推論順序、責務分離、失敗時の振る舞い、継続条件を定義する境界オブジェクトとなる。

非構造的なプロンプトでは、複数の責務が一つの要求へ押し込められる。LLM は、前提を推定し、不足を補い、矛盾を検出し、安全性を判断し、解法を選び、最終回答を生成することを同時に期待される。LLM はテキストを継続するよう訓練されているため、入力が不十分、矛盾、または安全でない場合でも、もっともらしい回答へ進みやすい。

この問題は、単にモデル能力の不足ではない。プロンプト内に状態境界が存在しないことも大きな原因である。

Interface-Led Architecture (ILA) は、この問題に有用な抽象化規律を与える。ILA では、インタフェースを持つ構成要素を Provider、Observer、Operator という基本役割へ分類する。上位構造はそれらの合成体である Unit として扱われ、Unit は Core Contract の上で Pipeline によって連結される。

本稿は、この規律をプロンプト空間へ移植する。Prompt-State Machine は、プロンプトを Provider、Observer、Operator の状態へ分離し、Clarify と FailClosed を明示的な失敗時遷移として組み込む構造化プロンプトである。さらに最小例に加え、利用可能な推論能力を Provider state 内で宣言し、それを Observer が検証してから Operator が使用する Skill-Set Provider パターンも提示する。

2. ILA / AIKernel 体系における位置づけ

本稿は、ILA 本体論文および Provider-Observer-Operator 抽象化規律を補足する technical note である。理論的な系譜は次の通りである。

Interface-Led Architecture:

インタフェース中心のソフトウェア設計方法論。

Provider-Observer-Operator:

インタフェースを抽出するための役割ベース抽象化規律。

Prompt-State Machines:

その抽象化規律を LLM プロンプト空間へ適用したモデル。

本稿は、ILA の一般理論そのものを拡張するものではない。また、AIKernel における Semantic IR、Composite Distance、Governed Circuit、ReplayLog などの特殊化を、そのままプロンプト空間へ持ち込むものでもない。

本稿の位置づけは、Provider-Observer-Operator によって定義された ILA の役割ベース抽象化規律を、LLM プロンプト設計へ応用することである。すなわち、Prompt-State Machine は ILA の新しい基本構成要素ではなく、ILA の抽象化規律をプロンプト空間へ適用した応用モデルである。

この区別は重要である。

ILA general theory:

Role / Unit / Pipeline / Core Contract.

AIKernel specialization:

Semantic IR / Composite Distance / ReplayLog / Governed Circuit.

Prompt-State Machine:

Prompt-level role sequencing and failure transitions.

3. Problem Statement

典型的なプロンプトは、モデルに直接回答を求める。

この問題を解いてください。

このようなプロンプトでは、LLM は以下を暗黙に実行しなければならない。

- 前提を特定する
- 不足している文脈を推定する
- 矛盾を検出する
- 曖昧性を分類する
- 解法を選ぶ
- 質問すべきかを判断する
- 回答して安全かを判断する
- 最終回答を生成する

しかし、これらは同じ責務ではない。それぞれ異なるアーキテクチャ上の役割に対応する。

前提を列挙する前に回答するモデルは、Provider 境界を持たない。不足情報を観測する前に解法を組み立てるモデルは、Observer 境界を持たない。曖昧性が残っているにもかかわらず処理を継続するモデルは、失敗時遷移を持たない。

Prompt-State Machine は、推論を次の状態列として構造化する。

Provider -> Observer -> Operator

重要な規則は、Observer が不足情報、矛盾、安全でない実行意図、または統治不能な曖昧性を検出した場合、Operator へ進んではならない、という点である。その場合、Prompt-State Machine は Clarify または FailClosed へ遷移する。

4. Prompt-State Machine Model

Prompt-State Machine (PSM) とは、状態、役割責務、遷移順序、失敗経路を定義する構造化プロンプトである。

本稿では、PSM を次のように表す。

PSM = (M, S, T, R, F)

ここで、

- M は YAML front matter など表現されるメタ情報である。
- S は状態集合である。
- T は状態遷移関係である。
- R は各状態に割り当てられた責務である。
- F は Clarify や FailClosed などの失敗時遷移である。

最小モデルでは、状態集合は次のようになる。

S = { Provider, Observer, Operator, Clarify, FailClosed }

通常の遷移経路は次の通りである。

Provider -> Observer -> Operator -> Answer

ガード付きの遷移経路は次の通りである。

Provider -> Observer -> Clarify

Provider -> Observer -> FailClosed

これは形式オートマトン理論における厳密な決定論的状态機械ではない。LLM は依然として確率的生成器である。本稿で定義する Prompt-State Machine は、推論順序を制約し、失敗時の遷移を明示するための、プロンプトレベルの疑似ステートマシンである。

5. Provider State

Provider state は、推論に必要な材料を供給する状態である。前提、利用可能な情報、制約、既知の事実、能力境界を明示する。

Provider は最終回答を生成しない。また、入力が曖昧か安全でないかを評価しない。Provider の責務は、利用可能な材料を明示することである。

典型的な Provider 指示は次の通りである。

Provider

- 問題を解くために必要な前提を列挙せよ
- ユーザーが明示的に与えた情報を整理せよ
- 利用可能な制約、データ、資源を特定せよ
- 必須情報が不足している場合は Unknown として記録せよ
- 必須情報を推測してはならない

Provider state の価値は、モデルが仮定を最終回答の中へ隠すことを防ぐ点にある。モデルが「何を与えられていると考えているか」を外部化するための状態である。

6. Observer State

Observer state は、Provider によって供給された材料を観測する。曖昧性、必須情報の不足、矛盾、危険な仮定、安全でない実行意図、根拠のない主張を検出する。

Observer は最終回答を生成しない。また、観測状態を勝手に解法へ変形してはならない。Observer は、Operator へ進んでよいかを判断するための証拠を提供する。

典型的な Observer 指示は次の通りである。

Observer

- 入力に矛盾がないか観測せよ
- 曖昧な用語または未定義の条件を検出せよ
- 安全な回答に必要な不足情報を列挙せよ
- 推測してよい仮定と推測してはいけない仮定を分離せよ
- 明確化が必要な場合は質問を生成せよ
- 安全でない、または統治不能な要求は FailClosed へ遷移せよ

この状態は、Prompt-State Machine における最も重要な安全境界である。すべてのプロンプトを回答可能なものとして扱うことを防ぐ。

7. Operator State

Operator state は、変換、推論、合成、計画、または最終回答生成を行う状態である。Provider が材料を供給し、Observer が処理継続可能であると判断した後にのみ動作する。

典型的な Operator 指示は次の通りである。

Operator

- Provider の出力と Observer の観測結果を用いて解法を構築せよ
- 必須情報を捏造してはならない
- Observer が Clarify を要求している場合、最終回答を生成するな
- 安全に支持できる範囲内でのみ回答せよ
- 安全な完了が不可能な場合は Clarify または FailClosed を返せ

Operator は、最終回答を生成できる唯一の状態である。この分離により、モデルの推論経路は監査しやすくなり、レビュー可能性も高まる。

8. Clarify and FailClosed Transitions

Prompt-State Machine は、すべての入力を回答へ接続しない。

Clarify は、処理継続に必要な情報をユーザーへ質問する遷移である。FailClosed は、安全または意味のある処理を継続できない場合に、処理を停止する遷移である。

典型的な遷移規則は次の通りである。

```
Observer detects missing required information -> Clarify
Observer detects contradiction -> Clarify
Observer detects unsafe execution request -> FailClosed
Observer cannot verify required assumptions -> Clarify or FailClosed
```

これはプロンプトレベルにおける fail-closed design の対応物である。不確実性の下で推測する代わりに、モデルは停止し、質問し、または拒否するよう構造化される。

9. Markdown / ROM as State Boundary Representation

Markdown は、Prompt-State Machine にとって実用的な状態境界表現である。

その利点は、単なる見た目ではなくアーキテクチャ上の性質にある。

- 見出しによって状態を分離できる。
- YAML front matter によってメタ情報を定義できる。
- 箇条書きによって責務を表現できる。
- コードブロックによって入出力形式を固定できる。
- 人間が構造を容易にレビューできる。
- LLM が構造に従いやすい。

ROM 的な Markdown は、プロンプトを非構造文字列ではなく、役割セクション、関係ヒント、制約、失敗時の振る舞いを持つ統治された文書として扱うことで、このパターンをさらに強化する。

本稿は Markdown が唯一の状態境界表現であると主張するものではない。XML、JSON、YAML、DSL、schema-enforced tool calls などとも状態を表現できる。本稿が Markdown を重視する理由は、人間可読性と LLM 可読性のバランスが実用上優れているためである。

10. Minimal Example

以下は、最小構成の Prompt-State Machine である。

```
---
task: " 問題解決"
pipeline:
  - provider
  - observer
  - operator
fail_closed: true
clarify_when:
  - missing_required_information
  - contradiction_detected
  - unsafe_execution_request
---

# Provider
```

- 問題を解くために必要な前提を列挙せよ
- ユーザーが明示的に与えた情報を整理せよ
- 不足している情報があれば Unknown として明示せよ
- 必須情報を推測してはならない

Observer

- 入力の矛盾、曖昧さ、不足を観測せよ
- 推測してよい仮定と推測してはいけない仮定を分離せよ
- 明確化が必要ななら質問を生成せよ
- 明確化が必要な場合は Operator へ進むな

Operator

- Provider の出力と Observer の観測結果から解法を構築せよ
- Observer が Clarify を要求している場合、最終回答を生成するな
- 安全に支持できる範囲でのみ最終回答を生成せよ

同じ考え方は英語でも表現できる。

```
---
task: "problem solving"
pipeline:
  - provider
  - observer
  - operator
fail_closed: true
---
```

Provider

- List assumptions required to solve the problem.
- Organize information explicitly provided by the user.
- Mark missing information as Unknown.

Observer

- Inspect the input for contradictions, ambiguity, and omissions.
- Separate safe assumptions from assumptions that must not be guessed.
- If clarification is required, ask and do not proceed to Operator.

Operator

- Construct the solution using Provider and Observer outputs.
- Do not generate a final answer if Observer requested Clarify.
- Generate the final answer only within the safely supported scope.

10.1 Skill-Set Provider Example

上記の最小例は、意図的に汎用的な形にしている。実用上は、Provider state において、入力情報だけでなく、そのタスクで利用してよいスキルセットを明示すると、Prompt-State Machine はより具体的で扱いやすくなる。

ここでいうスキルセットとは、モデルが当該タスクで利用してよい推論、変換、領域処理の能力を、プロン

プトレベルで宣言したものである。LLM は、要求内容から自分が使うべき能力を暗黙に推定しがちである。したがって、利用可能なスキルを明示しない場合、モデルは本来許可されていない能力や、未確認の前提を黙って仮定する可能性がある。

Skill-Set Provider は、Observer と Operator が進む前に、利用可能な推論資源とその境界を明示するための Provider パターンである。

これは、Prompt-State Machine の一般構造に対する一つの特例化である。一般モデルにおける Provider は推論材料を供給する。これに対して Skill-Set Provider では、Provider が利用可能な能力境界も供給する。すなわち、どのスキルを使ってよいか、各スキルがどの入力が必要とするか、どの仮定が禁止されるか、どの出力契約を守るべきかを明示する。この設計により、ILA の一般的な役割モデルを損なわず、実用的なプロンプト境界をより明確にできる。

実用的な Skill-Set Provider は、次のような項目を持つ。

Field	意味
skill_id	スキルの安定識別子。
purpose	そのスキルが何を支援してよいか。
allowed_operations	Operator が利用してよい操作。
required_inputs	そのスキルを使う前に必要な入力。
forbidden_assumptions	推測してはならない前提。
output_contract	スキル出力の期待構造。
fallback	必須情報が不足する場合の遷移先。

以下は、技術的な問題解決タスクにおける Skill-Set Provider の例である。

```
---
task: "technical problem solving"
pipeline:
  - provider
  - observer
  - operator
fail_closed: true
skill_set:
  - skill_id: requirements_extraction
    purpose: " 明示された要求、制約、不明点を抽出する "
    allowed_operations:
      - list_requirements
      - identify_constraints
      - mark_unknowns
    required_inputs:
      - user_request
    forbidden_assumptions:
      - 明示されていない要件を作ること
      - 隠れた業務ルールを仮定すること
    output_contract: "requirements, constraints, unknowns"
    fallback: clarify

  - skill_id: solution_design
    purpose: " 検証済みの Provider / Observer 出力だけに基づいて解決案を構築する "
    allowed_operations:
      - propose_solution
      - compare_options
      - explain_tradeoffs
```

```

    required_inputs:
      - requirements
      - observer_findings
    forbidden_assumptions:
      - 不足情報を既知の情報として扱うこと
      - Observer の警告を無視すること
    output_contract: "solution, rationale, limitations"
    fallback: fail_closed
  ---

# Provider

- `skill_set` セクションを読み込め
- このタスクで利用可能なスキルを列挙せよ
- 各スキルについて required_inputs と forbidden_assumptions を列挙せよ
- 必須入力不足しているスキルは unavailable として明示せよ

# Observer

- 各 required_inputs が存在するか確認せよ
- forbidden_assumptions を使わなければ進めない箇所を検出せよ
- 安全に利用できないスキルがある場合、Clarify または FailClosed へ遷移せよ

# Operator

- Provider が available とし、Observer が検証したスキルのみを使用せよ
- required_inputs が不足しているスキルを使用してはならない
- 宣言された output_contract に従って回答を生成せよ

```

この例は、Provider が能力情報を運ぶ方法を示している。ただし、これはプロンプトを完全なランタイムシステムへ変えるものではない。スキルセットは AIKernel Core Contract のように実行を強制するものではなく、モデルが前提としている推論資源を可視化し、観測可能で、統治可能にするためのプロンプトレベルの構造である。

設計原則は単純である。スキルは、使用される前に宣言され、信頼される前に観測され、必要入力と境界が検証された後にのみ Operator によって使用されるべきである。

11. SGP および Semantic Runtime Theory との関係

Prompt-State Machine は、AIKernel の SGP Pipeline および Semantic Runtime Theory と構造的に対応する。ただし、両者は同一ではない。

SGP は Structure、Generation、Polish を分離する。AIKernel Semantic Runtime Theory は、人間の意図を観測可能な意味状態として扱い、それを統治、再現、実行へ接続する。Prompt-State Machine は、同じアーキテクチャ的直観を、より軽量なプロンプトレベルで実現する。すなわち、責務と遷移が明示された推論は、より安全に扱える。

Prompt-State Machine	AIKernel Runtime / Semantic Runtime	対応する意味
Provider	ROM / VFS / Context Provider	推論に必要な前提、知識、制約、材料を供給する。
Observer	Admissibility Gate / Trajectory Observer / Risk Observer	曖昧性、不足情報、候補行動、リスク、安全でない仮定を観測する。

Prompt-State Machine	AIKernel Runtime / Semantic Runtime	対応する意味
Operator	Inference Operator / Transform Operator / Synthesis Operator	観測された証拠に基づき、推論、変換、回答生成、構造化を行う。処理継続に必要な情報が不足している場合、最終回答生成を止めて明確化を求める。
Clarify	Clarify disposition / User confirmation path	
FailClosed	Deny / Abort / Safe fallback	継続が安全でない、または統治不能な場合に、回答生成や実行を停止する。
Markdown / ROM-like Prompt	Structured Context / ROM boundary	人間と LLM の双方が読める状態境界表現を提供する。
Prompt-State Machine	Governed reasoning pattern	ランタイム強制ではなく、プロンプトレベルの軽量ガバナンスを提供する。

対応関係は次のように要約できる。

Prompt-State Machine:
Prompt-level governed reasoning pattern.

AIKernel Governed Circuit:
Runtime-level governed execution topology.

Prompt-State Machine は Governed Circuit の簡易実装ではない。これは、統治された実行アーキテクチャのプロンプト空間における対応物である。

12. Chain-of-Thought / ReAct との関係

Chain-of-Thought prompting は、中間推論ステップを明示させることで、複雑な推論課題に対する性能が向上し得ることを示した。ReAct は、推論の痕跡と行動ステップを交互に組み合わせ、モデルが推論し、行動し、観測し、計画を更新する形式を示した。

Prompt-State Machine はこれらのアプローチと関連するが、目的が異なる。

本稿の目的は、単に長い推論過程を引き出すことでも、行動と観測を交互に並べることでもない。本稿の目的は、プロンプト自体にアーキテクチャ上の責務分離を与えることである。

- Provider は材料を供給する。
- Observer は処理を継続してよいかを観測する。
- Operator は観測後にのみ操作する。
- Clarify と FailClosed は安全でない継続を防ぐ。

したがって、Prompt-State Machine は、性能向上のためのプロンプト技法というよりも、ガバナンス指向のプロンプト構造として理解されるべきである。

13. Limitations and Non-Claims

本稿は、プロンプト構造だけで LLM の内部推論過程を完全に制御できると主張するものではない。LLM は確率的生成器であり、プロンプトレベルの構造はカーネルレベルの強制とは同等ではない。

本稿は、Markdown や ROM が唯一の Prompt-State 表現であるとも主張しない。環境によっては他の構造化形式の方が適切な場合もある。

Prompt-State Machine は、AIKernel Runtime Governance、Pre-Inference Admissibility Governance、Trajectory Governance、Semantic Runtime Theory を置き換えるものではない。より弱い強制条件の下で同じ設計直観を反映する、プロンプトレベルのガバナンスパターンである。

最後に、本稿は中間推論のすべてをエンドユーザーへ露出すべきだと主張するものではない。Prompt-State Machine は内部的に推論を構造化し、外部には必要な最終回答、明確化要求、または拒否だけを提示する形で利用できる。

14. Conclusion

本稿は、Interface-Led Architecture および Provider-Observer-Operator 抽象化規律を LLM プロンプト設計へ応用し、Prompt-State Machine というモデルを定義した。

LLM は確率的生成器であるが、構造化プロンプトはその推論過程を明示的な状態と失敗時遷移へ誘導できる。Provider は前提と材料を供給し、Observer は曖昧性、矛盾、不足、リスクを検出し、Operator は観測結果を解法または回答へ変換する。Clarify と FailClosed は、不十分または安全でない入力が無視されて最終回答へ変換されることを防ぐ。

Markdown および ROM 的構造は、人間可読性、LLM 可読性、レビュー容易性を兼ね備えた実用的な状態境界表現である。

Prompt-State Machine は、AIKernel の SGP Pipeline、Trajectory Governance、Semantic Runtime Theory と構造的に対応しつつ、ランタイムで強制される回路ではなく、プロンプトレベルのガバナンスプロトコルとして機能する。

この意味で、ILA は実装空間だけでなく、プロンプト空間においても機能する。すなわち、推論を統治可能にする必要がある場所であれば、ILA は役割、境界、遷移を整理するための方法論を提供できる。

本稿で提示した構造は、複数モデルを用いた将来的な実験的検証によって、Prompt-State Machine の有効性と再現性を評価する余地を残している。

References

1. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.”Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
2. Sogawa, Takuya. “Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture.”Zenodo, 2026. DOI: 10.5281/zenodo.20322690.
3. Sogawa, Takuya. “AIKernel Phase-2 Theory: Semantic Compilation Architecture.”Zenodo, 2026. DOI: 10.5281/zenodo.20312092.
4. Harel, David. “Statecharts: A Visual Formalism for Complex Systems.”Science of Computer Programming, vol. 8, no. 3, 1987, pp. 231-274. DOI: 10.1016/0167-6423(87)90035-9.
5. Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” arXiv:2201.11903, 2022.
6. Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. “ReAct: Synergizing Reasoning and Acting in Language Models.”International Conference on Learning Representations (ICLR), 2023.
7. CommonMark. “CommonMark: A Strongly Defined, Highly Compatible Specification of Markdown.”CommonMark Specification. Available at: <https://commonmark.org/>.